

Experiment - 01

Student Name: Sumir Malhotra
Branch: CSE - AIML
Semester: 5
Subject Name: ADBMS

UID: 24BAI70227
Section/Group: 24AIT_KRG-1_A
Date of Performance: 11/7/26
Subject Code: 24CSP-310

(A)

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. **"Low Salary"**: All monthly salaries strictly less than \$20,000.
2. **"Average Salary"**: All monthly salaries in the inclusive range [\$20,000, \$50,000].
3. **"High Salary"**: All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

Output Table	
category	accounts_count
Low Salary	1
Average Salary	1
High Salary	3

Solution:

```
1. SELECT
2.     'LOW CATEGORY' AS CATEGORY,
3.     COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
4. FROM ACCOUNTS
5. WHERE INCOME < 20000
6.
7. UNION ALL
8.
9. SELECT
10.    'AVERAGE CATEGORY' AS CATEGORY,
11.    COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
12. FROM ACCOUNTS
13. WHERE INCOME > 20000
14.    AND INCOME <= 50000
15.
16. UNION ALL
17.
18. SELECT
19.    'HIGH CATEGORY' AS CATEGORY,
20.    COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
21. FROM ACCOUNTS
22. WHERE INCOME > 50000;
```

(B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns of employees, department and salaries, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

Output Table:

name	dept_name	salary
Alice	HR	50000
Bob	HR	25000
Charlie	IT	80000
David	IT	40000
Emma	Sales	45000

Solution:

```
1. SELECT S.*, E.*, D.*
2. FROM EMPLOYEES AS E
3. JOIN DEPARTMENTS AS D ON E.DEPT_ID = D.DEPT_ID
4. JOIN SALARIES AS S ON S.EMP_ID = E.EMP_ID;
5.
6. UPDATE SALARIES
7. SET SALARY = SALARY + (SALARY * 0.10)
8. WHERE EMP_ID IN (
9.     SELECT E.EMP_ID
10.    FROM EMPLOYEES AS E
11.   JOIN DEPARTMENTS AS D ON E.DEPT_ID = D.DEPT_ID
12.  WHERE D.DEPT_NAME = 'HR'
13. );
14.
15. SELECT * FROM EMPLOYEES;
16.
17. SELECT EMP_ID
18. FROM SALARIES
19. WHERE SALARY > (SELECT AVG(SALARY) FROM SALARIES);
```

(C)

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza_toppings.

Every customer must choose **exactly 3 different toppings** to prepare a pizza.

Write a PostgreSQL query to generate **every possible 3-topping pizza combination** along with its **total ingredient cost**.

Requirements

Every pizza must contain **exactly 3 different toppings**.

The same topping **cannot appear more than once** in a pizza.

Different arrangements of the same toppings should be considered the **same pizza**.

Example:

Chicken, Pepperoni, Sausage

Pepperoni, Chicken, Sausage

Sausage, Chicken, Pepperoni

These are all the same pizza, so only **one** should appear.

Display the topping names separated by commas.

Display the total ingredient cost of each pizza.

Sort the output:

First by **highest total cost**

If two pizzas have the same cost, sort them alphabetically.

Solution:

```
1. SELECT CONCAT(P.TOPPING_NAME, ',', P2.TOPPING_NAME, ',', P3.TOPPING_NAME) AS PIZZA,  
2.      (P.INGREDIENT_COST + P2.INGREDIENT_COST + P3.INGREDIENT_COST) AS TOTAL_COST  
3. FROM PIZZA_TOPPING AS P  
4. JOIN PIZZA_TOPPING AS P2 ON P.TOPPING_NAME < P2.TOPPING_NAME  
5. JOIN PIZZA_TOPPING AS P3 ON P2.TOPPING_NAME < P3.TOPPING_NAME  
6. ORDER BY TOTAL_COST DESC;
```

(D)

Amazon wants to identify **returning customers** who made another purchase shortly after their first purchase. Write a PostgreSQL query to find all users who made **another purchase within 1 to 7 days** of an earlier purchase.

Requirements

1. Compare purchases **made by the same user only**.
2. Ignore comparisons where the same transaction is matched with itself.
3. The second purchase must occur **after** the first purchase.
4. Same-day purchases must **not** be considered.
5. The second purchase must occur within **7 days** of the first purchase.
6. Display only the **unique User IDs** that satisfy the above conditions.
7. Sort the output in ascending order of user_id.

Output Table

user_id

101

103

Solution:

```
1. SELECT DISTINCT T1.USER_ID  
2. FROM AMAZON_TRANSACTIONS AS T1  
3. JOIN AMAZON_TRANSACTIONS AS T2 ON T1.USER_ID = T2.USER_ID  
4. WHERE T2.CREATED_AT::DATE > T1.CREATED_AT::DATE  
5.    AND T2.CREATED_AT::DATE - T1.CREATED_AT::DATE <= 7  
6. ORDER BY T1.USER_ID ASC;  
7.
```